

Эффективная работа с удаленными запросами. Shared future

Наибольший прирост производительности можно получить, распараллеливая удаленные запросы к БД. Для этого в Wasaby Framework существует механизм shared future (перевод с англ. в контексте: "делимый результат в будущем"), который предоставляет возможность работы с параллельными вычислениями на языках Python и C++.

Далее разобран пример использования механизма shared future для распараллеливания запросов. Рост производительности будет обеспечен за счёт сокращения времени ожидания ответа от сервера БД. Теперь оно формируется не из суммарного времени выполнения каждого запроса, а состоит из времени выполнения самого длительного запроса.

- Python. Работа с удаленными запросами производится через класс [sbis.BLObject](#). Для использования технологии shared future подходит метод [FutureInvoke\(\)](#).

```
1 # Запрос выполняется 1 секунду.
2 result1 = BLObject( 'Тест' ).FutureInvoke( 'Запрос1' )
3
4 # Запрос выполняется 2 секунды.
5 result2 = BLObject( 'Тест' ).FutureInvoke( 'Запрос2' )
6
7 # Запрос выполняется 3 секунды.
8 result3 = BLObject( 'Тест' ).FutureInvoke( 'Запрос3' )
```

Метод возвращает специальный объект типа [sbis.FutureObject](#). Чтобы получить результат работы метода, используют метод [get\(\)](#).

```
1 # Обрабатываем результаты.
2 return result1.get().Size() + result2.get().Size() +
   result3.get().Size()
```

- C++. Работа с удаленными запросами производится через класс [bl::Object](#). Для использования технологии shared future подходит метод [Invoke](#).

```
1 // Запрос выполняется 1 секунду.
2 auto result1 = bl::Object( L"Тест" ).Invoke< sbis::shared_future<
   RecordSet > >( L"Запрос1" ),
3
4 // Запрос выполняется 2 секунды.
5 result2 = bl::Object( L"Тест" ).Invoke< sbis::shared_future<
   RecordSet > >( L"Запрос2" ),
6
7 // Запрос выполняется 3 секунды.
8 result3 = bl::Object( L"Тест" ).Invoke< sbis::shared_future<
   RecordSet > >( L"Запрос3" );
9 return result1.get().Size() + result2.get().Size() +
   result3.get().Size();
10
```

В итоге суммарное время ожидания ответа составит 3 секунды вместо 6.

Особенности использования механизма shared future

- Метод `get()` возвращает результат, если он уже вычислен. В ином случае будет ожидать завершения вычисления побочного потока.
- Не рекомендуется использовать механизм shared future для небольших запросов. Эффективность можно получить при удалённых запросах, такие как запросы к БД. В C++ можно повысить производительность, используя настоящую многопоточность — выполнять вычисления, используя все ядра процессора.

- Рекомендуется производить другие вычисления перед вызовом результатов, пока ожидается ответ от сервера.
- Все параметры будут передаваться в другой поток по значению, т.е. будут копироваться. Поэтому если в C++ передается указатель, нужно позаботиться о потокозащищенности переменной, на которую ссылается указатель.
- Если при обработке запроса произошло исключение, то оно будет проброшено в основной поток при попытке получения результата через метод `get`.
- В порождаемый поток будут передаваться все потокозависимые настройки, такие как стратегия выдачи сообщений об ошибках.
- За особенностями работы `future`-механизма можно обратиться к [документации по boost](#).

Ограничения механизма `shared future`

Первое ограничение

Для того чтобы обезопасить программу от чрезмерного размножения потоков (которое в итоге приводит к тому, что программа начинает создавать исключения `"boost_resource_error"` — сообщение о нехватке системных ресурсов), был введен механизм, ограничивающий количество потоков, запущенных через `FutureInvoke`.

Пример. Предположим, что получен массив данных, и необходимо параллельно обработать его.

```

1  std::vector<MyData> v = GetData();
2  std::vector< std::shared_future<bool> > results;
3  for( size_t i = 0; i < v.size(); ++i )
4
5      // Запускаем параллельную обработку полученных данных
6      results.push_back( Object(L"My").Invoke<sbis::shared_future<bool> >
7      ( L"Process", v[i] ) );
8  for( size_t i = 0; i < results.size(); ++i )
9      printf(L"%d\n", results[i].get());

```

В случае, если метод `GetData()` вернул большое количество данных, вызовется множество `FutureInvoke`, которые должны породить потоки. Если бы ограничений на число потоков, порожденных `FutureInvoke`, не существовало, то такой код при большом размере массива привел бы к "падению" сервиса.

В текущей реализации обработка производится следующим образом:

- Если количество выполняемых на данный момент времени потоков `FutureInvoke` не превышает значение лимита в конфигурациях (по умолчанию — 16 потоков, регулируется настройкой облака "Ядро.Сервер приложений.МаксимальноеКоличествоВнутреннихПотоков"), то вызов производится асинхронно.
- Если лимит превышен, то вызов производится синхронно, в вызывающем потоке: `FutureInvoke` в таком случае ведет себя аналогично обычному методу [Invoke\(\)](#).
- Если за время работы 17-го часть первых запросов отработала, то новые запросы уйдут в освободившиеся потоки. Если лимит будет снова превышен, то очередной вызов опять же будет исполнен синхронно.
- Если все вызовы работают одинаково, то получаем распараллеливание на 17 потоков (16 дополнительных + основной). Если работают по-разному, то теоретически возможна работа в один поток. Например, первые 16 запросов отработали за 1 секунду, а 17-ый запрос, который синхронный, "затупил" и работал 100 секунд. Получается, что 99 секунд работал только один поток, т.к. распределение по потокам происходит только после завершения синхронного вызова.

Второе ограничение

Если на момент выхода из метода бизнес-логики остались незавершенные вызовы, запущенные через `FutureInvoke`, то платформа дожидается их завершения перед отдачей ответа. Это опять же предохраняет сервис от чрезмерного размножения "висячих" потоков.

Пример, когда проявится данная особенность поведения платформы:

```

1  sbis::shared_future<bool> res =
2  Object(L"My").Invoke<sbis::shared_future<bool> >(L"Test");
3
4  if( !some_condition() )
5
6      // Поняли, что дальше по каким-то причинам обработка невозможна и

```

```
5     вышли, не дождавшись результата FutureInvoke
6     return false;
7     ...
```

Если исполнение пройдет по ветке `return false`, то механизм платформы после выхода из метода "притормозит" исполнение, дождется завершения `"My.Test"` и только после этого отдаст наружу ответ `false`.

Третье ограничение

Метод вызванный через `FutureInvoke` выполняется в отдельном соединении с БД и в отдельной транзакции, соответственно ему недоступны изменения, сделанные в неподтвержденной транзакции основного потока.

Прикладной пример

Переменная `result` — это результат выполнения некоторого декларативного метода, который возвращает набор записей. Для каждой записи необходимо заполнить поля: `"РП.ПользователиНаКонтроле"`, `"РП.СвязанныеДокументы"` и `"РП.КонтрагентЕстьЛиКонтакты"`. Значение для каждого поля берётся из результата выполнения конкретного метода бизнес логики.

```
1  for one_doc in result:
2
3      # Первый запрос. Запускаем выполнение метода
      "ПользователиНаКонтроле" из объекта бизнес логики "СвязьПапок".
      Результат помещаем в поле "РП.ПользователиНаКонтроле".
4      one_doc.ПользователиНаКонтроле = СвязьПапок.ПользователиНаКонтроле(
      arr, filterRec, None, None )
5
6      # Второй запрос. Он подобен первому запросу, но будет запущен
      только после его завершения.
7      one_doc.СвязанныеДокументы =
      СвязьПапок.ПолучитьСвязанныеДокументыИТипы( int(one_doc['@Документ']) )
8
9      # Третий запрос. Он будет запущен только после завершения второго
      запроса.
10     one_doc.КонтрагентЕстьЛиКонтакты =
      СвязьПапок.КонтрагентЕстьЛиКонтакты( str(one_doc['Контрагент.ИНН']),
      str(one_doc['Контрагент.@Лицо']),
      str(one_doc['Контрагент.ИдентификаторСПП']) )
11
12     # В итоге поля записи будут заполнены за время, равное суммарному
      времени выполнения всех запросов.
```

Для ускорения заполнения полей записи необходимо применить технологию **shared future** — запускаем параллельное выполнение трёх методов бизнес-логики.

```
1  for one_doc in result:
2
3      # Необходимость расширения программного кода на три дополнительных
      строки обусловлена спецификой работы механизма shared future.
4      # "КонтрагентЕстьЛиКонтакты" – самый медленный метод. На его
      выполнение требуется больше времени по сравнению с двумя другими
      методами.
5      # "ПолучитьСвязанныеДокументыИТипы" является самым быстрым методом.
6      # Запускаем выполнение трёх методов в таком порядке, чтобы
      уменьшить "накладные расходы".
7      future_contacts = BLObject( 'СвязьПапок' ).FutureInvoke(
      'КонтрагентЕстьЛиКонтакты', str(one_doc['Контрагент.ИНН']),
```

```

1   str(one_doc[ 'Контрагент.@Лицо' ]),
   str(one_doc[ 'Контрагент.ИдентификаторСПП' ]) )
8   future_monitor = BLObject( 'СвязьПапок' ).FutureInvoke(
   'ПользователиНаКонтроле', arr, filterRec, None, None )
9   future_connect = BLObject( 'СвязьПапок' ).FutureInvoke(
   'ПолучитьСвязанныеДокументыИТипы', int(one_doc[ '@Документ' ]) )
10
11   # В данной точке программы get возвращает результат метода
   "ПолучитьСвязанныеДокументыИТипы".
12   # Если результат ещё не готов, то программа будет его ожидать.
   Другие строки кода выполняться не будут.
13   one_doc.СвязанныеДокументы = future_connect.get()
14   one_doc.ПользователиНаКонтроле = future_monitor.get()
15   one_doc.КонтрагентЕстьЛиКонтакты = future_contacts.get()
16
17   # Сначала запрашиваем результат выполнения самого быстрого метода
   "ПолучитьСвязанныеДокументыИТипы".
18   # В последнюю очередь запрашиваем результат метода
   "КонтрагентЕстьЛиКонтакты", т.к. он выполняется дольше двух других.

```

Эффективное распараллеливание ограниченного набора задач

Основной недостаток механизма FutureInvoke при распараллеливании набора задач - невозможность контролировать на каком потоке будет выполняться вызываемый метод. Например, если обработка запускается в цикле, может получиться так, что "долгая" задача находится в середине набора и блокирует выполнение более быстрых задач находящихся в конце. Для решения этой проблемы предназначен класс ParallelTasks.

Задачи добавляются в объект ParallelTasks с помощью методов AddTask и AddInvoke. AddTask добавляет произвольный функтор, AddInvoke - локальный вызов RPC метода. Когда все задачи добавлены, их нужно запустить методом Launch. Launch запускает и дожидается их выполнения. Запущенные задачи выполняются на тех же потоках, что и вызовы FutureInvoke, а также на потоке вызвавшем Launch. В отличие от FutureInvoke освободившиеся потоки будут получать новые задачи даже когда текущий поток занят.

Примеры

```

1   // Запуск трех функторов при помощи ParallelTasks
2   auto func1 = []() -> int { return int{ 5 }; };
3   auto func2 = []() -> String { return L"Hello, world"_s; };
4   auto func3 = []() -> void { return ; };
5
6   blcore::ParallelTasks pt;
7   auto f1 = pt.AddTask( func1 );
8   auto f2 = pt.AddTask( func2 );
9   auto f3 = pt.AddTask( func3 );
10  pt.Launch();
11  // f1.Get() == 5
12  // f2.Get() == L"Hello, world"_s;
13
14  // Запуск трех функторов при помощи LaunchParallelTasks
15  auto func1 = []() -> int { return int{ 5 }; };
16  auto func2 = []() -> String { return L"Hello, world"_s; };
17  auto func3 = []() -> void { return ; };
18
19  auto [ f1, f2, f3 ] = blcore::LaunchParallelTasks( func1, func2, func3
  );

```

```
20 // f1.Get() == 5
21 // f2.Get() == L"Hello, world"_s;

1  pt = sbis.ParallelTasks();
2  # добавление задач
3  pt.AddTask( lambda: 42 )
4  pt.AddTask( lambda: "hello, world" )
5  pt.AddInvoke('Тест.Эхо', 'test')
6  # запуск на выполнение, вызов блокируется до окончания выполнения всех
   задач
7  futures = pt.Launch()
8  # futures[0].Get() == 42
9  # futures[1].Get() == "hello, world"
10 # futures[2].Get() == "test"
```